1. Python More on Strings

Strings, Unicode, and Formatting in Python

In Python 3, strings are Unicode by default, allowing support for international languages, special symbols, and emojis. This overcomes older ASCII limitations, which only supported a small set of characters.

String manipulation can be challenging due to dynamic typing, especially when combining strings

with numbers. Instead of manually converting values using str(), Python provides a string formatting mini-language that uses placeholders inside strings.

Using the format() method, values can be inserted into strings by position or by name, making code cleaner and more readable. String formatting also supports advanced features such as controlling decimal precision, alignment, sign display, and scientific notation.

String handling and formatting are essential skills for data cleaning, and these features are heavily used

in assignments throughout the course. The next step is applying these concepts while reading and writing structured data files.

2. Python Demonstration: Reading and Writing CSV files(ipynb)

Iterating Through CSV Data and Computing

Summary Statistics

This tutorial demonstrates how to read and analyze CSV data in Python using the built-in csv module. A CSV file is read with csv.DictReader, converting each row into a dictionary where column names are keys and values are strings. The dataset contains 234 rows, each representing a car.

Basic summary statistics are computed by iterating through the list of dictionaries. To calculate averages such as city or highway MPG, values must be converted from strings to floats before

performing arithmetic.

Grouped summaries are created by first identifying unique category values using a set, such as the number of cylinders or vehicle class. For each category, the code iterates through the dataset, accumulates totals, counts entries, and computes averages. Results are stored in a list and sorted for easier interpretation.

The examples show that city MPG decreases as cylinder count increases, and that pickup trucks have the lowest highway MPG while compact cars have the highest. This

approach illustrates fundamental data summarization through iteration, which will later be simplified and optimized using the Pandas library.

3. Python Dates and Times (ipynb)

Working with Dates and Times in Python

Dates and times are common in data analysis tasks such as aggregating sales, filtering events by period, or analyzing activity over

time. This course introduces only the basics of date and time handling in Python.

Many systems store dates as offsets from the Unix epoch (January 1, 1970), often as seconds or milliseconds. These numeric timestamps must be converted to readable date and time values for analysis.

Python provides the time and datetime modules to work with dates. You can obtain the current time since the epoch and convert it into a datetime object, which exposes attributes like year, month,

day, hour, and second.

datetime objects support arithmetic using timedelta, allowing subtraction, comparison, and the creation of sliding time windows. This is useful for tasks such as analyzing activity across fixed periods. More advanced date and time handling will be covered later using pandas.

4. Advanced Python Objects, map()(ipynb)

Object-Oriented and Functional Concepts in Python

Python supports object-oriented programming, though in data science it is more common to use existing objects than to create new classes. Classes are defined with the class keyword, use camel case naming, and rely on indentation for scope. Variables are created dynamically, and methods are defined like functions but must include self to access instance data. Objects are instantiated by calling the class name. Python does not

enforce private or protected members, and all attributes and methods are accessible. Constructors are optional and can be defined using __init__, but are not required.

Functional programming ideas are also important in Python, especially for data cleaning and analysis. While Python is not purely functional, it often uses functions as parameters and emphasizes chaining operations with minimal side effects.

The map function is a key functional feature. It applies a function to one

or more iterable inputs and returns a map object. This object uses lazy evaluation, meaning values are only computed when accessed. Maps are iterable and memory-efficient, which is useful when working with large datasets.

Both object-oriented and functional patterns appear frequently in Python data science workflows, even if they are used implicitly rather than explicitly defined.

5. Advanced Python

Lambda and List Comprehensions(ipynb)

Lambda Functions and List Comprehensions in Python

Lambda functions are anonymous, single-expression functions used for short, simple operations. They are defined using the lambda keyword followed by parameters, a colon, and one expression. The expression is evaluated and returned when the lambda is called.

Lambdas return a function reference, have no name, do not support default parameters, and cannot contain complex logic or multiple statements. They are limited compared to regular functions but are very useful for small data-cleaning tasks and are commonly seen in Python data science code.

Python also provides list comprehensions, a compact way to create lists from iterable sequences. They combine the loop, value expression, and optional condition into a single line, making code

shorter and often faster than traditional for-loops.

List comprehensions and lambdas emphasize concise, readable transformations and are widely used in data science workflows, even though their use is optional in this course.